

Aurora Architecture v1.2

Complete Instruction Reference and Baseline Timing Table

Canonical ISA Draft 1 - August 2026

This document defines the programmer-visible behavior, flags, exceptions, issue resources, baseline latency, initiation interval, mode interactions, and examples for the Aurora v1.2 instruction families. Exact binary encodings are defined in the companion Instruction Formats and Opcode Map specification.

1. Timing and Issue Conventions

- Latency is the number of core cycles from issue until the architectural result is available for a dependent instruction through forwarding or writeback.
- Initiation interval (II) is the minimum cycles between issuing operations to the same functional resource when operands and destinations are independent.
- Slot A favors scalar ALU, address, control, compare, branch, and simple register-layout work. Slot B favors multiply, divide, packed-wide DSP, reduction, stream, and memory operations.
- The four-entry pairing window may pair the oldest instruction with one independent younger non-barrier instruction. It does not change architectural order.
- Memory latency assumes ITCM/DTCM or single-cycle on-chip SRAM. External-memory latency is implementation- and target-dependent and is scoreboarded.
- Multi-cycle instructions have bounded baseline timing but may complete later when waiting on memory, streams, or shared resources.

1.1 Flag notation

Z = zero N = negative/sign C = carry/borrow/last-shifted bit V = signed overflow
Q = sticky DSP status where defined - = unchanged

1.2 Mode notation

S32 = scalar 32-bit P16 = packed 2 x 16 per register / 4 x 16 per pair
P8 = packed 4 x 8 per register / 8 x 8 per pair L16/L8 = low-lane masked
P64 = paired 64-bit scalar emulation

2. Consolidated Baseline Timing Table

Family	Slot	Latency	II	Comment
ADD/SUB/ADC/SBC	A	1	1	All ALU modes
AND/OR/XOR/ANDN	A	1	1	All ALU modes
CMP/TST	A	1	1	No destination
LSL/LSR/ASR/ROL/ROR S32/P8/P16	A	1	1	Lane-local
LSL/LSR/ASR/ROL/ROR P64	A/B	2-3	1	Scoreboarded pair
MUL 8/16 packed	B	2-4	1-2	Full products
MUL 32 x 32	B	4-8	2	Pair result
DIV/DIVSTEP	B	variable bounded	1	Multi-cycle
DOT/SAD	B	3-5	1-2	Reduction pipeline
REDUCE.ADD/logic	B	2-3	1	A0/A1 result
REDUCE.MIN/MAX/ ABSMAX	B	2-4	1	Comparator tree

Family	Slot	Latency	II	Comment
PACK/UNPACK/ZIP/UNZIP/BSWAP/WSWAP	A	1	1	Strong pass-through candidate
LD/ST scalar TCM	B	1-2	1	Memory scoreboard
LD.P/ST.P TCM	B	2	1-2	Atomic pair
Branch predicted correctly	A	1	1	Backward-taken
Branch mispredict	A	2-3	1	Flush/refill
CALL/RET	A	1-2	1	R13 link
PUSH/POP scalar	B	1-2	1	R14 implicit
QPEEK	B	1	1	No consume
QPOP/QPUSH	B	1-2	1	May wait on queue
SETMODE/QMASK	A	1	1	State barrier
MRS/MSR	A	1-2	1	Special register
ASC entry	A	implementation	-	Privileged transfer

3. Scalar and Packed ALU Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
ADD	Rd,Ra,Rb	Rd=Ra+Rb	ZNCV	A	1	1	-	S32/P16/P8/L16/L8/P64*	P64 is multi-cycle pair form
ADC	Rd,Ra,Rb	Rd=Ra+Rb+C	ZNCV	A	1	1	-	S32/L16/L8/P64*	Used for multiword arithmetic
SUB	Rd,Ra,Rb	Rd=Ra-Rb	ZNCV	A	1	1	-	All ALU	Borrow convention defined by C
SBC	Rd,Ra,Rb	Rd=Ra-Rb-(1-C)	ZNCV	A	1	1	-	S32/L16/L8/P64*	Multiword subtract
AND	Rd,Ra,Rb	Bitwise/lane AND	ZN--	A	1	1	-	All ALU	L8/L16 preserve upper bits
OR	Rd,Ra,Rb	Bitwise/lane OR	ZN--	A	1	1	-	All ALU	
XOR	Rd,Ra,Rb	Bitwise/lane XOR	ZN--	A	1	1	-	All ALU	
ANDN	Rd,Ra,Rb	Ra & ~Rb	ZN--	A	1	1	-	All ALU	Not a reduction operator
CMP	Ra,Rb	Set flags from Ra-Rb	ZNCV	A	1	1	-	All ALU	No destination
TST	Ra,Rb	Set flags from Ra & Rb	ZN--	A	1	1	-	All ALU	No destination
MIN	Rd,Ra,Rb	Signed/unsigned lane min	ZN--	A	1	1	-	S32/P16/P8/L16/L8	Signedness from mode
MAX	Rd,Ra,Rb	Signed/unsigned lane max	ZN--	A	1	1	-	S32/P16/P8/L16/L8	Signedness from mode

3.1 Low-lane write masking

L8: Rd = (old_Rd & 0xFFFFFFF0) | result[7:0]

L16: Rd = (old_Rd & 0xFFFF0000) | result[15:0]

To avoid a third register-file read port, low-lane arithmetic is preferably encoded as a two-address operation in which Rd is also source A. The scoreboard remains register-granular.

4. Immediate, Shift, and Rotate Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
ADDI	Rd,Ra,#imm	Rd=Ra+sext(i mm)	ZNCV	A	1	1	-	S32/L16/L8	Extension word for long immediate
ANDI/ORI/XORI	Rd,Ra,#imm	Logic with immediate	ZN--	A	1	1	-	S32/L16/L8	
MOVI	Rd,#imm	Load immediate	ZN--	A	1	1	-	S32	Long form uses 0xE escape
LSL	Rd,Ra,#n	Logical left shift	ZNC-	A	1	1	-	All	P64: 2-3 cycles
LSR	Rd,Ra,#n	Logical right shift	ZNC-	A	1	1	-	All	
ASR	Rd,Ra,#n	Arithmetic right shift	ZNC-	A	1	1	-	S32/P16/P8/L16/L8/P64	P64 sign is bit 63
ROL	Rd,Ra,#n	Rotate left	ZNC-	A	1	1	-	All	Count modulo active width
ROR	Rd,Ra,#n	Rotate right	ZNC-	A	1	1	-	All	Count modulo active width
SHR.R	Rd,Ra,#n	Rounded arithmetic right shift	ZNCQ	A/B	1-3	1	-	S32/L16/L8/P64	Optional profile if encoding retained

In paired-64 mode, even register fields name 64-bit pairs, while SP and PC may be selected as zero-extended source operands. Pair shifts reuse 32-bit hardware and are intentionally multi-cycle.

5. Multiply and Divide Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
MUL	Rd,Ra,Rb	Full-width signed/unsigned product	ZN-Q	B	2-8	1-2	Illegal pair	P8/P16/S32/P64	Mode selects lane/product width
MULH	Rd,Ra,Rb	High half of scalar product	ZN-Q	B	3-6	2	-	S32	If retained by final opcode map
MAC	A#,Ra,Rb	A=A+product/reduced products	---Q	B	3-6	1-2	-	P8/P16/S32	Accumulator select bit
MAS	A#,Ra,Rb	A=A-product	---Q	B	3-6	1-2	-	P8/P16/S32	Separate from REDUCE fold model
DIV	Rd,Ra,Rb	Integer quotient	ZNVQ	B	bounded multi	1	Divide by zero	S32	No scaled-divide opcode
DIVSTEP	state,operand	One restoring/nonrestoring step	ZN-Q	B	1	1	-	S32	Compiler/runtime building block

6. DSP Reduction, DOT, and SAD Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
DOT	A#,Ra,Rb	A=sum(lane products)	---Q	B	3-5	1-2	Stream underflow	P8/P16	Overwrite or fold
SAD	A#,Ra,Rb	A=sum(abs(Ra _i -Rb _i))	---Q	B	3-5	1-2	Stream underflow	P8/P16	Overwrite or fold
RED.ADD	A#,Ra	A=sum(lanes)	---Q	B	2-3	1	Stream underflow	P8/P16	Fold uses +
RED.AND	A#,Ra	A=AND(lanes)	---Q	B	2	1	Stream underflow	P8/P16	Fold uses AND

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
RED.OR	A#,Ra	A=OR(lanes)	---Q	B	2	1	Stream underflow	P8/P16	Fold uses OR
RED.XOR	A#,Ra	A=XOR(lanes)	---Q	B	2	1	Stream underflow	P8/P16	Fold uses XOR
RED.MIN	A#,Ra	A=min(lanes)	---Q	B	2-4	1	Stream underflow	P8/P16	Fold uses min(A,x)
RED.MAX	A#,Ra	A=max(lanes)	---Q	B	2-4	1	Stream underflow	P8/P16	Fold uses max(A,x)
RED.ABSMAX	A#,Ra	A=max(abs(lanes))	---Q	B	3-4	1	Min-negative policy	P8/P16	Fold uses max
RED.COUNT	A#,Ra	A=count(true lanes)	---Q	B	2-3	1	Stream underflow	P8/P16	Fold uses addition

6.1 Overwrite and fold semantics

overwrite: A := reduce(current chunk)

fold: A := combine(A, reduce(current chunk))

The combine operator is specific to the reduction. This permits unbounded stream reductions across repeated four-lane or eight-lane chunks without materializing intermediate scalars.

7. Register Layout and Shuffle Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
MOV	Rd,Rs	Full 32-bit copy	---	A	1	1	-	All	Ignores stream interception and L8/L16 mask
BSWAP	Rd,Rs	Reverse four bytes	ZN--	A	1	1	-	All	Strong pairing candidate
WSWAP	Rd,Rs	Swap low/high 16-bit halves	ZN--	A	1	1	-	All	
PACK	Rd,Ra,Rb	Pack selected low lanes	ZN-Q	A	1	1	-	All	Register-only
UNPACK	Rd,Rs	Widen selected lanes	ZN-Q	A	1	1	-	All	Register-only
ZIP	Rd,Ra,Rb	Interleave lanes	ZN--	A	1	1	-	P8/P16	
UNZIP	Rd,Rs	Deinterleave lanes	ZN--	A	1	1	-	P8/P16	
SHUF	Rd,Rs,#pattern	Byte/word permutation	ZN--	A	1-2	1	Illegal pattern	P8/P16/S32	Extended pattern if needed

These operations are unaffected by low-lane write masking so software can move upper stored bytes or halfwords into lane zero. They are non-faulting, register-only operations and are preferred pass-through candidates in the four-entry issue window.

8. Memory Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
LD.B/LD.H/LD.W	Rd,[Ra+off]	Load and extend	ZN--	B	1-2+	1	Align/bus/protection	S32	Signedness by subfunction
ST.B/ST.H/ST.W	Rs,[Ra+off]	Store low 8/16/32 bits	---	B	1-2+	1	Align/bus/protection	S32	Stores remain ordered
LD.P	Re,[Ra+off]	Atomic 64-bit pair load	ZN--	B	2+	1-2	Align/bus/protection	P64	Low word at lower address

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
ST.P	Re,[Ra+off]	Atomic 64-bit pair store	----	B	2+	1-2	Align/bus/protection	P64	R12:R13 conditional
LDA	Rd,[Ra+off]	Address calculation only	ZN--	A	1	1	-	S32/P64	No memory access

- Loads may pass unrelated register-only work but do not pass older stores in the v1.2 baseline.
- MMIO accesses are ordering barriers and never reordered.
- TCM accesses are deterministic. General SRAM and external-memory accesses use ready/ack and remain scoreboarded.
- Unaligned accesses may trap or be handled by ASC according to implementation profile.

9. Branch, Call, Loop, and Stack Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
B.cc	target	Conditional PC-relative branch	----	A	1 / 2-3	1	Target/protection	S32	Backward branches predicted taken
JMP	target/Rs	Unconditional transfer	----	A	1-2	1	Target/protection	S32	
CALL	target/Rs	R13=return PC; PC=target	----	A	1-2	1	Target/protection	S32	Clobbers LR
RET	R13	PC=R13	----	A	1-2	1	Target/protection	S32	May be branch alias
LOOPSET	count,target	Configure low-overhead loop	----	A	1	1	Privilege/profile	S32	Exact state in loop/AGU class
PUSH	Rs/list	SP-=size; store	----	B	1-2+	1	Stack/bus	S32/P64	R14 implicit
POP	Rd/list	load; SP+=size	----	B	1-2+	1	Stack/bus	S32/P64	

10. Stream and Queue Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
QMASK	#mask	Enable default Q0/Q1 interception	----	A	1	1	-	All	State barrier
QPEEK	Rd,Q#	Read current element, no advance	ZN--	B	1	1	Empty queue	P8/P16/S32/P64	
QPOP	Rd,Q#	Consume current element	ZN--	B	1-2+	1	Empty queue	P8/P16/S32/P64	Retirement advances state
QPUSH	Q#,Rs	Produce element	----	B	1-2+	1	Full queue	P8/P16/S32/P64	
QDUP	Q#	Duplicate local/LIFO top	----	B	1	1	Invalid top/full	All	
QREPL	Q#,Rs	Replace top/head	----	B	1	1	Invalid top	All	Occupancy unchanged
QCFG	Q#,fields	Configure address/stride/count/width	----	A/B	1-3	1	Privilege/config	All	Extension words likely
QON/QOFF	Q#	Enable/disable stream engine	----	A	1	1	-	All	One-cycle state change

Stream-capable DOT, SAD, REDUCE, and memory forms contain a consume/local override. Register-only layout instructions never consume streams. Address, count, and circular-wrap state advance only after successful retirement.

11. System, Timer, Interrupt, and ASC Instructions

Mnemonic	Operands	Semantics	Flags	Slot	Latency	II	Faults	Mode	Notes
MRS	Rd,SReg	Read special register	ZN--	A	1-2	1	Privilege	S32	Includes coarse TIME_COUNT
MSR	SReg,Rs	Write special register	----	A	1-2	1	Privilege	S32	State barrier
SETMODE	#mode	Set arithmetic interpretation	----	A	1	1	Illegal mode	All	Public ABI restores S32
TIMERLD	Rs/vector	Atomically load countdown and vector	---	A/B	1-2	1	Privilege	S32	Countdown in coarse ticks
IRET	-	Restore interrupt PC/PSR	all	A	2+	-	Privilege/frame	S32	Ordered privileged return
EI/DI	-	Enable/disable interrupt recognition	----	A	1	1	Privilege	S32	
BARRIER	class	Order memory/stream/system effects	----	A	1+	-	-	S32	Issue-window barrier
ASC	service	Enter Architecture Service Code	----	A	impl.	-	Privilege/service	S32	Protected ROM/ITCM transfer

11.1 Timer behavior

- TIME_COUNT is a 32-bit free-running counter of coarse timer ticks and wraps modulo 2^{32} .
- TIMER_COUNT is a single 32-bit one-shot countdown. Privileged software multiplexes all logical timers onto the next expiration.
- The implementation may use ripple, segmented, serial, or other low-cost counter circuitry behind a prescaler.
- Expiration-to-interrupt posting latency is fixed and documented; cycle-exact scheduling is not required.

12. Exceptions, Ordering, and Precise State

- Illegal opcode, illegal mode, odd or prohibited pair destination, divide by zero, alignment, bus, memory-protection, privilege, stream underflow/overflow, and ASC service faults are precise.
- Potentially faulting operations may execute for multiple cycles but cannot allow younger architectural state to retire past them.
- Simple non-faulting unary/register operations may bypass older stalled instructions when dependency and barrier checks pass.
- Stores, MMIO, stream configuration, mode changes, privileged operations, and unresolved control transfers are ordering barriers.
- The implementation may maintain small per-window completion records; this is not a general speculative reorder buffer.

13. Compiler Scheduling Model

- The compiler is the primary scheduler; hardware dynamic pairing repairs short local gaps within four decoded instructions.
- Alternate A0/A1 to hide accumulator feedback when kernel semantics permit.
- Pair scalar address/control work in Slot A with DSP, stream, multiply, reduction, or memory work in Slot B.
- Prefer BSWAP, WSWAP, PACK, UNPACK, ZIP, UNZIP, MOV, compare, and simple shifts as pass-through fillers.
- Model Q0/Q1 consumption, A0/A1 access, pair aliases, LR liveness, memory ports, writeback paths, and functional-unit initiation intervals.
- Use L8/L16 upper-lane storage conservatively; hazards remain whole-register granular.
- Use pair loads/stores for 64-bit spills and do not assume extra pair-only scratch registers.

14. Worked Scheduling Examples

14.1 Streaming DOT with dual accumulators

```

DOT      A0, Q0, Q1      ; initialize first chunk
ADDI     R6, R6, #8      ; Slot A work can pair
DOT.ACC  A1, Q0, Q1      ; alternate feedback chain
CMP      R6, R7
DOT.ACC  A0, Q0, Q1
B.LT     loop

```

14.2 Low-byte scalar storage

```

; R4 contains four packed byte temporaries
BSWAP    R4, R4          ; move a selected upper byte toward lane zero
ADD.L8   R4, R5          ; update low byte, preserve upper 24 bits
BSWAP    R4, R4          ; restore layout

```

14.3 Paired-64 fixed-point extraction

```

MUL      R2:R3, R4, R6   ; full product
ASR.P64  R2:R3, #15      ; multi-cycle paired shift
MOV      R0, R2          ; scaled low 32-bit result

```

14.4 Four-entry dynamic pairing

Window: [DOT A0,Q0,Q1 | DOT.ACC A0,Q0,Q1 | WSWAP R6 | ADDI R7,#1]
Issue: oldest DOT in Slot B + independent WSWAP in Slot A
Remain: [DOT.ACC A0,Q0,Q1 | ADDI R7,#1] in original order

15. Remaining Freeze Items

Item	Status	Action
Exact flag convention for subtraction C/borrow	Open	Must be uniform across SUB, SBC, CMP.
Saturation and rounding subset	Open	Current baseline avoids broad saturation; optional rounded shift requires final decision.
Exact multiply and divide latency by implementation profile	Profile	Architecture provides bounded classes; RTL synthesis sets numbers.
Exact stream underflow behavior	Open	Trap, stall, or programmable behavior must be finalized per queue mode.
Exact branch displacement and extension rules	Encoding	Defined in opcode map but requires final bit-level audit.
Instruction-by-instruction binary encodings	Next	Generate machine-readable ISA database and assembler tables.
Silicon-validated timing	Next	Replace provisional latency values after cycle-accurate RTL and synthesis.